N047 Cloud Run에서 AI서비스 운영하기_학습서

Chapter01

Cloud Run에서 AI 서비스 운영하기

에이전트 서비스가 공개 주소에서 돌아가기 시작하면 질문이 바뀝니다. "올라갔나?"가 아니라 "계속 잘 돌아가나?"를 묻게 됩니다. 답이 길어지면 중간에 끊기지는 않을까, 누가 주소를 퍼뜨려 요청이 몰리면 요금은 얼마나 나올까, 새 버전을 올렸더니 어제까지 대화하던 캐릭터가 사라졌다면 어디를 봐야 할까. 이런 질문은 서비스가 사용자를 만나는 순간부터 매일 생깁니다.

일반 웹앱과 달리 AI 서비스는 답이 느리고, 요청마다 LLM 제공사(OpenAI, Google처럼 LLM API를 제공하는 회사)에 돈을 내며, 같은 요청에도 처리 시간이 들쭉날쭉합니다. 그래서 같은 Cloud Run 위에서도 요청 타임아웃, 인스턴스 수, 사용 한도를 AI 서비스에 맞게 따로 정해야 합니다. 요청 타임아웃(request timeout)은 요청 하나를 처리할 수 있는 최대 시간이고, Cloud Run 콘솔에서는 "요청 제한 시간"으로 표시됩니다. 오늘은 실습용 에이전트 서비스에 이 설정을 하나씩 걸어 보고, 무슨 일이 있었는지를 로그에서 찾아 읽는 방법을 익힙니다.

주관식 퀴즈

Q. [시작 질문] 내가 만든 AI 서비스의 공개 주소를 단체 대화방에 올렸다고 상상해 보세요. 그날 밤 서비스에 어떤 일이 생길 수 있을지, 걱정되는 일을 세 가지 적어 보세요.

Chapter02

시작 상태 맞추기

오늘 실습은 에이전트 서비스 `friend-agent` 가 Cloud Run에 배포되어 있고, LLM API 키를 Secret Manager의 시크릿 `llm-api-key` 에서 읽는 상태에서 시작합니다. 시크릿(secret)은 API 키처럼 밖에 드러나면 안 되는 값이고, Google Cloud 콘솔에서는 "보안 비밀"로 표시됩니다. 이미 이 상태라면 아래 확인 명령만 실행하고 넘어갑니다. 처음 시작한다면 템플릿 저장소의 실습별 시작 상태 문서 A, B 순서대로 먼저 준비합니다. Google Cloud 가입부터 해야 한다면 30~50분쯤 걸립니다.

직접 실행할 명령: 시작 상태 확인

```
# 1. gcloud 계정, 프로젝트, 기본 리전
gcloud config list

# 2. friend-agent 서비스와 서비스 URL
gcloud run services describe friend-agent --region us-central1 --format "value(status.url)"

# 3. 시크릿 llm-api-key 가 있는지 (값은 출력하지 않는다)
gcloud secrets describe llm-api-key --format "value(name)"
```

처음부터 준비하는 경우의 순서는 다음과 같습니다. 각 단계의 정확한 명령은 시작 상태 문서에 있습니다.

1. Google Cloud 무료 체험 가입, 프로젝트 만들기, gcloud 설치와 `gcloud init` (시작 상태 문서 A)
2. 기본 리전 `us-central1` 설정과 API 4개 사용 설정 (A)
3. 템플릿 저장소로 내 공개 저장소를 만들고 내려받기 (A)
4. 콘솔의 Secret Manager에서 시크릿 `llm-api-key` 만들기. 키 값은 콘솔에서 직접 넣는다 (B)
5. 기본 서비스 계정에 그 시크릿 읽기 권한을 주고, 시크릿을 참조해 `friend-agent` 배포 (B)

배포가 끝나면 서비스 URL(배포하면 Cloud Run이 붙여 주는 공개 주소)을 브라우저로 열어 캐릭터를 만들고 한마디 걸어 봅니다. 답이 오면 준비가 끝난 것입니다.

코딩 에이전트에게 맡길 때: 시작 상태 확인

```
gcloud에 설정된 프로젝트와 리전, friend-agent 서비스의 서비스 URL, 시크릿 llm-api-key 가 있는지 확인해 줘.
시크릿의 값은 읽거나 출력하지 마. 확인만 하고 아무것도 만들거나 바꾸지 마.
```

Chapter03

스트리밍과 요청 타임아웃

LLM은 답을 앞에서부터 조금씩 만들어 냅니다. 다 만들어질 때까지 기다렸다가 한꺼번에 보내면 사용자는 빈 화면을 오래 보게 되고, 만들어지는 대로 조금씩 보내는 스트리밍을 쓰면 전체 시간은 같아도 첫 글자가 바로 보입니다. 템플릿의 에이전트 서비스는 메시지를 받는 `/api/chat` 을 스트리밍으로 만들어 두었습니다. 그런데 이 요청은 답을 다 보낸 뒤에도 바로 끝나지 않습니다. 같은 요청 안에서 방금 대화에서 기억할 것을 정리해 저장하기 때문입니다. Cloud Run은 요청 하나가 요청 타임아웃(기본 300초)을 넘으면 그 지점에서 연결을 끊습니다.



그림: 메시지 하나를 처리하는 순서와 요청 타임아웃에 걸린 두 경우

그림의 둘째 줄처럼 답을 보내는 중에 끊기면 "내일 서울은 맑다가" 같은 문장 중간에서 멈추고, 뒤에 오던 비 소식이 잘려 맑은 날로 읽힐 수 있습니다. 셋째 줄처럼 기억 정리 중에 끊기면 답은 끝까지 보이는데 방금 말한 일정은 저장되지 않을 수 있습니다. 요청 타임아웃을 짧게 잡았을 때 생기는 더 큰 문제는 이렇게 틀리거나 빠진 결과가 정상처럼 보이는 것입니다. 템플릿의 클라이언트(client), 곧 브라우저에서 돌아가는 화면 코드는 서버가 스트리밍을 마칠 때 보내는 스트림 종료 이벤트가 오지 않으면 "응답이 끝까지 오지 않았어요"라는 안내를 붙이도록 만들어 두었는데, 이런 확인이 없는 클라이언트라면 사용자는 잘린 답을 그대로 믿습니다.

배포한 서비스에서 직접 확인해 봅니다. 화면 오른쪽 "이번 턴" 패널의 걸린 시간 칸에서 첫 글자, 답 전체, 요청 전체가 각각 몇 초였는지 먼저 읽습니다. 로컬 환경(내 컴퓨터)에서 잰 예시로는 요청 전체가 3.5~4.5초쯤이었습니다. 요청 타임아웃을 요청 전체보다 짧은 3초로 줄인 뒤 몇 번 말을 걸어, 답이 중간에 멈추는 경우와 답은 끝까지 왔지만 안내가 붙는 경우를 찾아보세요. 내 서비스에서 잰 시간이 다르면 줄일 값도 그에 맞춰 바꿉니다.

직접 실행할 명령: 요청 타임아웃 줄였다가 되돌리기

```
gcloud run services update friend-agent --region us-central1 --timeout 3
```

```
# 확인한 뒤 기본값으로 되돌린다
gcloud run services update friend-agent --region us-central1 --timeout 300
```

코딩 에이전트에게 맡길 때: 요청 타임아웃 줄이기

```
friend-agent 서비스의 요청 타임아웃을 3초로 줄여서 다시 배포해 줘. 다른 설정은 바꾸지 마.
```

직접 확인할 것: 요청 타임아웃을 3초로 줄인 상태에서 몇 번 말을 걸면, 답이 문장 중간에서 멈추거나 답 끝에 "응답이 끝까지 오지 않았어요" 안내가 붙습니다. 오른쪽 이번 턴 패널의 걸린 시간 칸에서 요청 전체 시간이 3초 근처에서 끊긴 것도 함께 봅니다. 몇 번 보내도 끊기지 않으면 처음에 읽은 요청 전체 시간보다 짧은 값으로 다시 줄입니다.

확인했으면 요청 타임아웃을 기본값 300초로 되돌립니다. 요청 타임아웃은 로그와 측정항목에서 가장 오래 걸린 요청의 처리 시간을 보고 그보다 넉넉하게 잡습니다. 요청 시간에는 답을 보내는 시간뿐 아니라 그 뒤의 기억 정리까지 들어간다는 점을 함께 계산합니다.

내 서비스의 처리 시간 읽기

요청 타임아웃을 정하는 근거는 짐작이 아니라 내 서비스가 실제로 걸린 시간입니다. 이번 턴 패널은 내가 보낸 메시지 하나의 시간만 보여 주므로, 여러 사람이 쓴 요청 전체는 측정항목 탭에서 봅니다. 요청 지연 시간 그래프에는 50번째, 95번째, 99번째 백분위수가 함께 나오는데, 95번째 백분위수는 요청 100건 가운데 95건이 이 시간 안에 끝났다는 뜻입니다. 요청 타임아웃은 가장 느린 쪽 값보다 넉넉하게 잡고, LLM 제공사의 API가 느려지는 날에는 이 값이 몇 배로 늘 수 있다는 점도 함께 생각합니다.

직접 확인할 곳: 요청 처리 시간 보기

1. 콘솔 맨 위 프로젝트 선택 버튼에 실습 프로젝트가 보이는지 확인하고, 검색창에 `Cloud Run` 을 입력해 서비스 목록을 엽니다. 바로가기 주소는 `https://console.cloud.google.com/run?project=<프로젝트 ID>` 입니다.
2. `friend-agent` 를 누르고 위쪽의 측정항목(Metrics) 탭을 누릅니다.
3. 탭 위쪽의 기간을 1시간이나 6시간으로 줄여 방금 보낸 요청이 보이게 합니다.
4. 요청 지연 시간(Request latencies) 그래프를 찾고, 그래프 위쪽의 백분위수 선택에서 50번째와 99번째를 차례로 골라 값을 읽습니다. 그래프 선 위에 마우스를 올리면 시각별 값이 나옵니다.
5. 두 값을 섹션 앞에서 정한 요청 타임아웃과 비교합니다.

Chapter04

인스턴스 수 제한

서비스 URL은 누구나 호출할 수 있고, 에이전트 서비스는 요청마다 내 키로 LLM API 비용이 나갑니다. 그래서 인스턴스 수에 상한을 겁니다. 최대 인스턴스는 요청이 몰려도 켜지는 인스턴스의 상한이고, 최소 인스턴스는 요청이 없어도 켜 두는 수입니다. 최소를 1로 두면 콜드 스타트는 사라지지만 요청이 없어도 요금이 계속 나가므로, 오늘은 최소 0, 최대 2로 정합니다.

직접 실행할 명령: 인스턴스 수 설정

```
gcloud run services update friend-agent --region us-central1 --min-instances 0 --max-instances 2
```

코딩 에이전트에게 맡길 때: 인스턴스 수 설정

```
friend-agent 서비스의 최소 인스턴스는 0, 최대 인스턴스는 2로 설정해서 다시 배포해 줘. 다른 설정은 바꾸지 마.
```

설정한 뒤 10분쯤 서비스를 쓰지 않다가 다시 말을 걸어 보세요. 첫 답이 늦게 시작되는 것이 콜드 스타트이고, 측정항목 탭의 인스턴스 수 그래프에서 0으로 내려갔다가 1로 올라오는 구간도 찾을 수 있습니다. 설정이 들어갔는지는 리비전 탭에서 맨 위 리비전 이름을 누르면 열리는 오른쪽 패널에서 확인합니다.

직접 확인할 곳: 인스턴스 수 그래프

1. Cloud Run > `friend-agent` > 측정항목(Metrics) 탭을 엽니다.
2. 컨테이너 인스턴스 수(Container instance count) 그래프를 찾습니다. 그래프가 여러 개라 아래로 내려야 보일 수 있습니다.
3. 기간을 1시간으로 두고, 쉬었던 10분 동안 선이 0에 붙어 있다가 다시 말을 건 시각에 1로 올라오는 구간을 찾습니다. 선이 실행 중(active)과 유휴(idle)로 나뉘어 보이면 둘을 합친 값이 인스턴스 수입니다.

사라진 캐릭터와 대화

오늘 요청 타임아웃을 바꾸고 인스턴스 수를 바꿀 때마다 새 리비전이 배포되었습니다. 배포한 뒤 화면을 다시 열면 만들어 둔 캐릭터와 나눈 대화가 사라지고 캐릭터 만들기 화면이 다시 나왔을 것입니다. 콜드 스타트를 확인하려고 10분 쉬었다가 돌아왔을 때도 같은 일이 생깁니다. 화면 왼쪽 아래의 저장소 표시에 적힌 대로, 템플릿은 지금 캐릭터와 대화와 기억을 서버(인스턴스) 메모리에 저장하기 때문입니다.

새 리비전은 새 인스턴스로 실행되고, 요청이 없어 인스턴스가 0개로 줄면 그 안의 메모리도 함께 지워집니다. 최대 인스턴스를 2로 둔 지금은 다른 문제도 생길 수 있습니다. 요청이 몰려 인스턴스가 둘이 되면 같은 사람의 메시지가 서로 다른 인스턴스로 갈 수 있고, 인스턴스마다 따로 기억을 들고 있게 됩니다. 컨테이너는 언제든 꺼지고 새로 켜질 수 있다는 전제로 쓰는 실행 단위이므로, 남겨야 하는 데이터는 컨테이너 밖의 데이터베이스에 둬야 합니다.

객관식 퀴즈

Q. 에이전트 서비스의 요청 타임아웃만 바꿔 다시 배포했더니, 방금까지 대화하던 캐릭터가 사라지고 캐릭터 만들기 화면이 나왔습니다. 가장 알맞은 설명은 무엇인가요?

- 1. 요청 타임아웃을 바꾸면 Cloud Run이 저장된 데이터를 일부러 지운다
- 2. 새 리비전은 새 인스턴스에서 실행되고, 캐릭터와 대화는 이전 인스턴스의 메모리에만 있었다
- 3. 브라우저가 캐시를 지워서 화면이 처음 상태로 돌아갔다
- 4. 최대 인스턴스가 2라서 데이터를 2개까지만 보관한다**제출**

- •

  1. 요청 타임아웃을 바꾸면 Cloud Run이 저장된 데이터를 일부러 지운다

- •

  2. 새 리비전은 새 인스턴스에서 실행되고, 캐릭터와 대화는 이전 인스턴스의 메모리에만 있었다

- •

3. 브라우저가 캐시를 지워서 화면이 처음 상태로 돌아갔다

- •

4. 최대 인스턴스가 2라서 데이터를 2개까지만 보관한다

콜드 스타트를 줄이는 방법과 대가

콜드 스타트가 싫다면 최소 인스턴스를 1로 둘 수 있습니다. 요청이 없어도 인스턴스 하나가 켜져 있으니 첫 답이 늦어지지 않지만, 그동안의 사용 시간에 요금이 계속 붙습니다. 수업용 서비스처럼 요청이 드문 서비스라면 몇 초 늦는 첫 답을 받아들이고 최소 0을 두는 편이 낫고, 사용자가 많아 늘 인스턴스가 켜져 있는 서비스라면 최소 인스턴스의 추가 비용이 상대적으로 작아집니다. 이미지 크기를 줄이거나 앱이 시작할 때 하는 일을 줄여 콜드 스타트 자체를 짧게 만드는 방법도 있는데, 이미지 크기와 시작 작업을 줄이는 방법은 컨테이너 이미지를 직접 만들 때 다룹니다.

Chapter05

최대 인스턴스로 막지 못하는 비용

최대 인스턴스 2개면 비용 걱정은 끝났을까요. 아래 화면의 ① 자동 호출이 몰릴 때 탭에서 확인합니다. 인스턴스 하나가 동시에 처리하는 요청 수를 동시성(concurrency)이라고 하며, Cloud Run 기본값은 80입니다. 처음 값이 오늘 설정(최대 인스턴스 2, 동시성 80, 답변 8초)이고, 답변 하나의 LLM API 비용은 예시 값입니다. 최대 인스턴스를 100개(기본값)로 바꿔 보고, 월 사용 한도를 "걸지 않음"으로도 바꿔 보세요.

최대 인스턴스 2개로도 동시에 160건을 처리해 한 시간에 7만 건이 넘는 답이 만들어지고, 예시 원가로는 시간당 100달러를 넘습니다. 최대 인스턴스는 한 번에 처리하는 양을 제한할 뿐, 시간이 흐르며 쌓이는 호출 수는 막지 않습니다. 총액을 막는 것은 LLM 제공사 쪽 월 사용 한도입니다. 한도에 닿으면 LLM 제공사가 호출을 거부해 서비스는 답을 못 하게 되지만 비용은 정한 금액을 넘지 않습니다.

이 과정에서 쓰는 키는 회사가 발급해 나눠 준 키라서, 월 사용 한도와 알림도 회사의 LLM 제공사 계정에 걸려 있고 수강생 화면에서는 볼 수도 바꿀 수도 없습니다. 같은 이유로 한도에 닿으면 내 서비스뿐 아니라 같은 키를 쓰는 다른 수강생의 서비스도 함께 답을 못 하게 됩니다. 내 키로 서비스를 운영하게 되면 LLM 제공사 대시보드의 결제 또는 사용량 메뉴에서 월 한도를 필요한 만큼만 작게 걸고, 한도의 절반쯤에서 알림이 오게 하는 것이 배포 전에 할 첫 작업입니다. 결제 정보가 있는 화면이므로 명령이나 에이전트로 하지 않고 직접 설정합니다.

객관식 퀴즈

Q. 에이전트 서비스에 최대 인스턴스 2를 설정했습니다. 이 설정에 대한 설명으로 옳은 것은 무엇인가요?

• 1. 동시에 처리되는 요청이 최대 2건으로 제한된다
• 2. 한 달 동안 LLM API 비용이 일정 금액을 넘지 않게 막아 준다
• 3. 켜지는 인스턴스가 2개를 넘지 않지만, 인스턴스마다 여러 요청을 동시에 처리하므로 LLM API 호출 총량을 제한하지는 않는다
• 4. 인스턴스가 항상 2개 켜져 있어서 콜드 스타트가 없어진다**제출**

- •

1. 동시에 처리되는 요청이 최대 2건으로 제한된다

- •

  2. 한 달 동안 LLM API 비용이 일정 금액을 넘지 않게 막아 준다

- •

  3. 켜지는 인스턴스가 2개를 넘지 않지만, 인스턴스마다 여러 요청을 동시에 처리하므로 LLM API 호출 총량을 제한하지는 않는다

- •

  4. 인스턴스가 항상 2개 켜져 있어서 콜드 스타트가 없어진다

요청 100건당 비용

Cloud Run은 요청을 처리하는 동안의 CPU와 메모리, 요청 수로 요금을 받고, LLM 제공사는 주고받은 토큰 수로 요금을 받습니다. 에이전트 서비스는 LLM API의 답을 기다리는 동안에도 요청이 열려 있어서 LLM API가 느릴수록 Cloud Run 요금도 늘어납니다. 메시지 하나에 LLM API를 세 번 호출한다는 점(도구 선택, 답, 기억 정리)도 LLM API 비용에 그대로 들어갑니다. `friend-agent` 의 측정항목 탭에서 요청 수와 요청 지연 시간(50번째 백분위수, 요청 절반이 이 시간 안에 끝났다는 뜻)을 읽습니다.

직접 확인할 것: 측정항목 탭의 기간을 오늘 실습한 시간대로 맞추면 요청 수, 요청 지연 시간, 컨테이너 인스턴스 수 그래프가 차례로 보입니다. 요청 지연 시간 그래프에서 50번째 백분위수를 골라 값을 읽어 두고, 아래 비용 계산에 씁니다.

읽은 지연 시간을 이 섹션 위쪽 화면의 ② 요청 100건당 비용 탭에 넣고, 토큰 수와 단가는 내가 쓰는 LLM 제공사의 요금표 값으로 바꿔 보세요. 토큰 수는 세 번의 호출을 합친 값으로 넣습니다. 처음 값으로 보면 요청 100건당 Cloud Run 비용은 2센트 정도이고, LLM API 호출 비용은 그보다 열 배 넘게 큽니다. Cloud Run 무료 사용량만으로 한 달에 2만 건 넘게 처리할 수 있어서, 수업 정도의 사용량이라면 실제로 돈이 나가는 곳은 LLM 제공사입니다. 이 계산은 요청 하나가 인스턴스를 혼자 쓴다고 가정한 값이라 실제 Cloud Run 요금은 더 적습니다.

Google Cloud 결제 메뉴의 예산 및 알림에서 예산을 걸어 둘 수도 있지만, 예산 알림은 메일을 보낼 뿐 지출을 멈추지 않습니다. 오늘 건 설정을 정리하면 아래와 같습니다.

설정	막는 것	막지 못하는 것
Cloud Run 최대 인스턴스	한 번에 처리하는 요청 수	시간이 흐르며 쌓이는 호출 수
LLM 제공사 월 사용 한도	한 달 LLM API 비용 총액	Cloud Run 쪽 요금
Google Cloud 예산 알림	없음 (알려 주기만 함)	지출 자체

사용자 키로 호출하는 BYOK 구조

지금 에이전트 서비스는 운영자의 키 하나로 모든 사용자의 요청을 처리합니다. 사용자가 늘면 LLM API 비용도 운영자가 모두 내고, 한도에 닿으면 모든 사용자가 함께 답을 못 받습니다. 그래서 사용자가 화면에서 자기 LLM API 키를 넣고, 서비스는 그 키로 LLM API를 호출하게 만드는 경우도 많은데, 이런 구조를 BYOK(Bring Your Own Key)라고 부릅니다. 비용은 사용자 계정으로 나가고 한도도 사용자가 정하지만, 대신 남의 키를 받아 다루게 됩니다. 코딩 에이전트에게 "사용자가 키를 넣게 해 줘"라고만 하면 키를 데이터베이스에 그대로 저장하거나 요청 로그에 남기는 코드가 나올 수 있으므로, 키를 어디에 두고 어디에 남기지 않을지를 먼저 정하고 그 구조대로 요청합니다.

키를 둘 곳은 서비스가 언제 LLM API를 호출하는지에 따라 정합니다.

구조	키가 있는 곳	맞는 서비스	대가
브라우저에 두고 요청마다 보내기	사용자 브라우저. 서버는 요청을 처리하는 동안만 메모리에서 쓰고 저장하지 않는다	사용자가 화면을 열어 둔 동안에만 LLM API를 호출하는 서비스	브라우저나 기기를 바꾸면 다시 넣어야 한다. 사용자가 없을 때 서버가 먼저 LLM API를 호출할 수 없다
서버에 암호화해 저장하기	서버 데이터베이스. 암호화에 쓰는 키는 Secret Manager	사용자가 화면을 닫은 동안에도 서버가 LLM API를 호출해야 하는 서비스	사용자를 구분하는 로그인, 암호화, 키 삭제 기능이 함께 필요하다. 데이터베이스와 암호화 키가 함께 새면 사용자 키도 드러난다

에이전트 서비스는 부탁받은 약속을 저장했다가 먼저 말을 거는 기능이 있어서, 사용자가 화면을 닫은 동안에도 알림을 보내려면 두 번째 구조가 필요합니다. 화면을 열었을 때만 대화하는 서비스라면 첫 번째 구조로 충분하고 만들기도 훨씬 쉽습니다. 서버를 거치지 않고 브라우저에서 LLM API를 바로 호출하는 방법도 있지만, 그러면 도구 선택이나 위기 표현 가드레일처럼 서버에서 하던 처리를 쓸 수 없습니다. 가드레일(guardrail)은 모델의 입력과 출력을 검사해 위험한 내용을 걸러 내는 처리입니다.

어느 구조를 고르든 아래는 함께 지킵니다.

- 요청 로그, 오류 메시지, "이번 턴" 패널에 키가 들어가지 않게 한다. 템플릿 작업 규칙의 시크릿 규칙이 사용자 키에도 그대로 적용된다.
- 키를 넣은 뒤에는 화면에 키 전체를 다시 보여 주지 않는다. 넣었는지 알 수 있게 끝 네 자리 정도만 보여 준다.
- 키가 틀렸거나 사용자 한도에 닿아 LLM 제공사가 호출을 거부하면, 그 이유를 사용자에게 알기 쉽게 안내한다. 이때 운영자 키로 몰래 대신 부르지 않는다.
- 사용자가 넣은 키를 지우는 버튼을 둔다.

운영자 키가 사라지는 것은 아닙니다. 처음 써 보는 사용자에게 몇 번은 운영자 키로 답해 주거나, 서비스가 스스로 하는 일에 운영자 키를 계속 쓰는 경우도 많고, 그 부분의 한도는 여전히 운영자가 관리합니다. 직접 만들어 보고 싶다면 섹션 7의 추가 과제에 요청 예시가 있습니다.

객관식 퀴즈

Q. 사용자가 화면에 자기 LLM API 키를 넣는 기능을 만들고 있습니다. 서버가 받은 요청을 디버깅하려고 요청 본문 전체를 로그에 남기도록 해 두었다면 어떤 문제가 생길까요?

• 1. 문제없다. 로그는 운영자만 볼 수 있다
• 2. 요청 본문에 들어 있는 사용자 키가 로그에 그대로 남아, 로그를 볼 수 있는 사람이면 누구나 그 키로 사용자 계정의 LLM API를 호출할 수 있다
• 3. Cloud Run이 키를 자동으로 가려서 남기므로 문제없다
• 4. 로그가 커져서 요금이 늘어나는 것만 문제다

- •

  1. 문제없다. 로그는 운영자만 볼 수 있다

- •

  2. 요청 본문에 들어 있는 사용자 키가 로그에 그대로 남아, 로그를 볼 수 있는 사람이면 누구나 그 키로 사용자 계정의 LLM API를 호출할 수 있다

- •

  3. Cloud Run이 키를 자동으로 가려서 남기므로 문제없다

- •

  4. 로그가 커져서 요금이 늘어나는 것만 문제다

Chapter06

로그의 세 종류

운영하는 동안 무언가 이상하면 가장 먼저 여는 곳이 로그입니다. 원인을 고치는 일은 코딩 에이전트와 함께 할 수 있지만, 무엇이 언제 어디서 일어났는지를 로그에서 찾아 근거로 건네는 일은 서비스를 운영하는 사람이 해야 합니다. 근거 없이 "안 돼요"라고만 하면 에이전트는 짐작으로 고치기 시작하고, 멀쩡한 곳을 바꾸기도 합니다.

Cloud Run의 로그 탭에는 남긴 주체가 다른 세 종류의 로그가 시간순으로 섞여 나옵니다. 요청 로그(request log)는 Cloud Run이 요청마다 자동으로 남기는 줄이고, 컨테이너 로그(container log)는 앱이 직접 출력한 줄이며, 시스템 로그(system log)는 인스턴스를 켜고 끄는 Cloud Run 플랫폼이 남기는 줄입니다. 빌드 중에 생긴 로그는 로그 탭이 아니라 Cloud Build 기록에 따로 남습니다.



그림: 로그 탭에 섞여 나오는 요청 로그, 컨테이너 로그, 시스템 로그와 로그 한 줄의 구조

에이전트 서비스는 컨테이너 로그를 JSON 한 줄로 남기고 `severity` 값을 붙입니다. Cloud Run은 이 값을 읽어 로그 화면의 심각도로 보여 주므로, 앱이 `ERROR` 로 남긴 줄은 심각도 필터에서 오류로 걸러집니다. 대화 내용 같은 원문은 남기지 않고, "기억 정리"처럼 무슨 일이 있었는지와 개수만 남기도록 만들어져 있습니다. 로그 한 줄에서 먼저 보는 것은 시각, 심각도, 리비전, 메시지 네 가지이고, 줄을 펼치면 나머지 필드가 보입니다. 리비전 칸이 중요한 이유는, 배포 직후에는 이전 리비전과 새 리비전의 로그가 함께 쌓이기 때문입니다.

직접 확인할 곳: 내 대화 한 번이 남긴 로그 찾기

1. 서비스 URL을 열고 캐릭터에게 한마디 보냅니다. 답이 끝나면 보낸 시각을 기억해 둡니다.
2. Cloud Run > `friend-agent` > 로그(Logs) 탭을 열고, 기간을 지난 1시간으로 둡니다.
3. 보낸 시각 근처에서 `POST` 와 `/api/chat` 이 있는 요청 로그를 찾습니다. 상태 코드(200)와 걸린 시간을 읽습니다.
4. 그 앞뒤에서 앱이 남긴 컨테이너 로그를 찾습니다. 예: "기억 정리". 줄 앞의 화살표를 눌러 펼친 뒤 `jsonPayload` 아래의 필드(예: `added`)를 확인합니다.
5. 로그 줄의 리비전 이름이 리비전 탭의 최신 리비전과 같은지 봅니다.

원하는 줄 찾기

요청이 많아지면 로그는 금방 수백 줄이 됩니다. 로그 탭에서는 심각도, 기간, 검색어로 거를 수 있고, 조건을 더 정확히 걸려면 로그 탐색기(Logs Explorer)를 씁니다. 로그 탐색기는 콘솔 검색창에 `로그 탐색기` 나 `Logs Explorer` 를 입력하거나 `https://console.cloud.google.com/logs/query?project=<프로젝트 ID>` 로 엽니다. 쿼리 입력 칸에 아래 같은 조건을 한 줄에 하나씩 적으면, 줄 사이는 모두 AND로 묶입니다.

찾는 것	쿼리
내 서비스의 로그만	`resource.type="cloud_run_revision"` 과 `resource.labels.service_name="friend-agent"`
오류 이상만	`severity>=ERROR`
서버 오류 응답	`httpRequest.status>=500`
3초보다 오래 걸린 요청	`httpRequest.latency>"3s"`
특정 리비전	`resource.labels.revision_name="friend-agent-00007-xyz"`
메시지에 특정 말이 든 줄	`jsonPayload.message:"기억 정리"`

`:` 는 "이 말이 들어 있다"는 뜻이고 `=` 는 "정확히 같다"는 뜻입니다. 위 두 줄(내 서비스의 로그만)을 먼저 적어 범위를 좁힌 뒤 나머지 조건을 더하면 결과가 빨리 나옵니다.

직접 확인할 곳: 세 가지 줄 찾아보기

1. 없는 주소 찾기: 서비스 URL 뒤에 `/nope` 을 붙여 엽니다. 로그 탐색기에서 `httpRequest.status=404` 로 그 요청 로그를 찾습니다.
2. 느린 요청 찾기: `httpRequest.latency>"3s"` 로 오늘 3초보다 오래 걸린 대화 요청을 찾고, 섹션 3에서 측정한 시간과 비교합니다.
3. 끊긴 요청 찾기: 섹션 3에서 요청 타임아웃을 3초로 줄였을 때 보낸 요청을 찾습니다. 요청 타임아웃에 걸린 요청은 504 같은 오류 상태 코드로 남습니다. `httpRequest.status>=500` 으로 찾아 상태 코드와 걸린 시간을 읽습니다.

직접 확인할 것: `/nope` 요청 로그를 찾았으면 줄 앞의 화살표를 눌러 펼칩니다. `httpRequest` 아래에 `status: 404` 와 지연 시간(`latency`)이 보이면, 쿼리가 제대로 걸린 것입니다.

직접 실행할 명령: 터미널에서 같은 로그 읽기

```
# 최근 로그 20줄을 콘솔과 비슷한 형식으로
gcloud run services logs read friend-agent --region us-central1 --limit 20

# 조건을 걸어 읽기: 지난 1시간 동안 friend-agent 의 오류 이상
gcloud logging read 'resource.type="cloud_run_revision" AND resource.labels.service_name="friend-agent"
AND severity>=ERROR' --freshness 1h --limit 20
```

객관식 퀴즈

Q. 사용자가 "어제 저녁에 답이 너무 늦게 왔어요"라고 알려 왔습니다. 가장 먼저 찾아볼 로그로 알맞은 것은 무엇인가요?

- 1. Cloud Build 기록의 빌드 로그
- 2. 어제 저녁 시간대의 요청 로그를 지연 시간 조건(`httpRequest.latency>"5s"` 같은)으로 거른 결과
- 3. 시스템 로그의 인스턴스 종료 기록만
- 4. 심각도가 ERROR인 컨테이너 로그만**제출**

- •

  1. Cloud Build 기록의 빌드 로그

- •

  2. 어제 저녁 시간대의 요청 로그를 지연 시간 조건(`httpRequest.latency>"5s"` 같은)으로 거른 결과

- •

  3. 시스템 로그의 인스턴스 종료 기록만

- •

  4. 심각도가 ERROR인 컨테이너 로그만

무슨 일이 있었는지 재구성하기

배포가 실패하면 그 흔적은 한 줄에 다 담기지 않고 여러 종류의 로그와 화면에 흩어져 남습니다. 그래서 흩어진 줄을 시간순으로 모아 무슨 일이 어떤 순서로 일어났는지 다시 세웁니다. 이번에는 코드를 고치지 않고 설정 하나만 틀리게 바꿔 실패하는 배포를 직접 만들어 봅니다. 에이전트 서비스는 `LLM_PROVIDER` 값이 `openai`, `gemini`, `claude`, `fake` 가 아니면 오류 로그를 남기고 시작하지 않습니다.

직접 실행할 명령: 일부러 실패하는 배포 만들고 되돌리기

```
# 1. LLM_PROVIDER 를 없는 값으로 바꿔 새 리비전을 만든다 (실패한다)
gcloud run services update friend-agent --region us-central1 --update-env-vars LLM_PROVIDER=unknown

# 2. 로그와 리비전 탭으로 타임라인을 적은 뒤, 원래대로 되돌린다
gcloud run services update friend-agent --region us-central1 --remove-env-vars LLM_PROVIDER
# 원래 gemini 나 claude 를 쓰고 있었다면 대신: --update-env-vars LLM_PROVIDER=<원래 값>
```

코딩 에이전트에게 맡길 때: 일부러 실패하는 배포 만들기

```
로그 읽기 연습을 하려고 해. friend-agent 서비스의 환경변수 LLM_PROVIDER 를 unknown 으로 바꿔서 배포해 줘.
실패할 거라는 걸 알고 하는 거니까 고치려고 하지 말고, 터미널에 나온 메시지만 그대로 보여 줘.
```

1번 명령은 실패로 끝나지만, 실패한 리비전은 트래픽을 받지 않으므로 서비스 URL은 이전 리비전으로 계속 답합니다. 브라우저에서 대화가 계속 되는지 먼저 확인하고, 그다음 아래 표를 채웁니다. 시각은 로그 줄 맨 앞의 시각을 그대로 옮깁니다.

시각	어디서 봤나	로그 종류	심각도	내용
	리비전 탭	해당 없음		새 리비전이 생겼고 상태가 실패로 표시됨
	로그 탭	컨테이너 로그		
	로그 탭	시스템 로그		
	로그 탭	요청 로그		같은 시각에 들어온 대화 요청은 어느 리비전이 처리했나
	터미널			배포 명령이 마지막에 출력한 메시지

표를 채우면 순서가 보입니다. 새 리비전이 만들어지고, 컨테이너가 켜지자마자 앱이 `LLM_PROVIDER` 오류를 남기고 꺼지고, 플랫폼이 시작 실패를 기록하고, 터미널에는 마지막 결과만 나옵니다. 터미널 메시지는 대개 이 순서의 끝, 곧 결과를 말하고, 원인은 그보다 앞선 컨테이너 로그 줄에 있습니다.

원인을 기록하는 방법

되돌린 뒤에는 무슨 일이 있었는지 아래 표 한 줄로 정리합니다. "설정 문제였음"처럼 줄이지 않고, 다음에 같은 메시지를 본 사람이 기록만 읽고 같은 줄을 찾을 수 있게 씁니다. 터미널 메시지는 본 그대로, 원인은 근거가 된 로그 줄과 함께 적습니다.

터미널 메시지	근거가 된 로그 줄	원인	조치
(터미널 마지막 메시지를 그대로)	컨테이너 로그 ERROR "LLM_PROVIDER 값 'unknown' 를 알 수 없어 ..." (리비전 이름과 시각)	환경변수 `LLM_PROVIDER` 에 지원하지 않는 값	환경변수를 지워 기본값으로 되돌림

에이전트에게 로그 건네기

원인을 고칠 때는 코딩 에이전트의 도움을 받아도 됩니다. 이때 에이전트에게는 짐작할 거리가 아니라 근거를 건넵니다. 무엇을 했고 무엇이 보였는지(증상), 언제였는지(시간 범위), 어느 서비스와 리비전인지, 내가 이미 찾은 로그 줄이 무엇인지 네 가지입니다. 로그 줄을 대화창에 붙여 넣기 전에는 API 키, 토큰, 사용자의 이름이나 연락처가 들어 있지 않은지 한 번 봅니다. 에이전트 서비스는 원문을 로그에 남기지 않지만, 직접 만든 다른 앱은 요청 내용을 그대로 남기기도 합니다.

로그를 직접 복사하지 않고, 에이전트가 위의 `gcloud logging read` 명령으로 로그를 읽게 할 수도 있습니다. 로그 읽기 명령은 아무것도 바꾸지 않으므로 승인해도 되는 명령이지만, 범위가 너무 넓으면 관계없는 줄까지 읽어 엉뚱한 결론을 내기 쉽습니다. 시간 범위와 서비스를 요청에 꼭 넣습니다.

코딩 에이전트에게 맡길 때: 로그로 원인 묻기

```
friend-agent 서비스에서 오늘 14시 00분부터 14시 10분 사이에 배포가 실패했어. 실패한 리비전은 friend-agent-00007-xy
z 야.
gcloud logging read 로 그 시간대의 이 서비스 로그만 읽고, 원인이라고 판단한 근거가 되는 로그 줄을 시각과 함께 그대로
보여 줘.
아직 아무것도 고치지 마.
```

에이전트가 근거로 든 로그 줄은 로그 탐색기에서 같은 시각과 메시지로 다시 찾아봅니다. 내가 찾은 줄과 같으면 원인을 믿고 고치는 단계로 넘어가고, 다르거나 찾을 수 없으면 에이전트에게 그 줄을 어디서 읽었는지 다시 묻습니다.

주관식 퀴즈

Q. [이해 점검] 위에서 채운 타임라인 표를 보고, 터미널 메시지만 보고 원인을 짐작했다면 어떤 오해를 할 수 있었을지, 실제 원인은 어느 로그 줄에서 확인됐는지 설명해 보세요.

Chapter07

서로 써 보기

옆 사람과 서비스 URL을 바꿔 들어가 봅니다. 상대 서비스에서 캐릭터를 하나 만들고 몇 마디 나눈 뒤, 내 서비스의 로그 탭을 엽니다. 내가 보내지 않은 요청이 내 서비스에 찍혀 있고, 측정항목 탭의 요청 수도 늘어 있습니다. 서비스 URL은 이렇게 누구나 쓸 수 있고, 그 요청마다 내 키로 LLM API 비용이 나갑니다. 섹션 4에서 건 인스턴스 수와 회사 계정에 걸린 사용 한도가 지켜 주는 것이 바로 이 상황입니다.

써 본 뒤에는 상대에게 들어갔을 때 첫 답이 늦었는지, 답이 끊긴 적은 없는지 한 줄씩 알려 줍니다. 들은 내용은 섹션 6에서 배운 방법으로 내 로그에서 확인해 봅니다. 늦었다는 시각의 요청 로그를 찾아 지연 시간을 읽으

면, 콜드 스타트였는지 LLM API 응답이 늦었는지 가늠할 수 있습니다.

서비스는 지금 인스턴스 메모리에 대화를 저장하므로, 상대가 만든 캐릭터와 내 캐릭터가 같은 사용자 칸에 섞일 수 있습니다. 로그인이 없는 지금 구조의 한계이고, 여러 사람이 쓰는 서비스라면 사용자를 구분해 데이터베이스에 저장해야 합니다.

직접 확인할 곳: 내 서비스에 남은 다른 사람의 요청

1. Cloud Run > `friend-agent` > 로그(Logs) 탭을 엽니다.
2. 로그 목록 위쪽의 검색 칸에 `/api/chat` 을 입력해 대화 요청 줄만 남깁니다.
3. 옆 사람이 들어온 시각 근처의 `POST /api/chat` 줄을 찾아 지연 시간을 읽습니다.
4. 측정항목(Metrics) 탭으로 옮겨 요청 수(Request count) 그래프에서 같은 시각에 선이 올라간 것을 확인합니다.

추가 과제

시간이 남으면 아래에서 하나를 골라 해 봅니다. 과제마다 필요한 안내가 그 과제 아래에 모두 있습니다.

과제	단계	하는 일
1. 문구를 바꿔 다시 배포하고 롤백하기	기본	오늘 한 배포와 트래픽 전환을 혼자 다시 해 본다
2. 부하 테스트로 동시성과 최대 인스턴스 확인하기	중간	동시 요청 50개를 보내 설정마다 인스턴스 수와 응답을 비교한다
3. 내가 만든 앱을 Cloud Run에 올리기	중간	다른 앱을 배포하고, 실패하면 로그로 원인을 찾는다
4. BYOK 구조로 바꾸기	도전	사용자가 자기 LLM API 키를 넣어 쓰게 만든다

과제 1. 문구를 바꿔 다시 배포하고 롤백하기

`static/index.html` 의 화면 문구나 `app/persona.py` 의 기본 캐릭터를 바꾸고 소스 배포합니다. 배포하기 전에 리비전 목록을 적어 두고, 새 리비전이 트래픽을 받는 것을 확인한 뒤 이전 리비전으로 롤백했다가 다시 최신으로 돌아옵니다. 리비전 탭에서 트래픽 비율이 바뀌는 것과, 롤백하는 동안 로그 탭의 요청 로그가 어느 리비전으로 찍히는지 확인하면 끝입니다.

직접 실행할 명령: 과제 1

```
gcloud run revisions list --service friend-agent --region us-central1
gcloud run deploy friend-agent --source . --region us-central1

# 이전 리비전으로 롤백했다가 최신으로 되돌리기 (리비전 이름은 목록에서 복사)
gcloud run services update-traffic friend-agent --region us-central1 --to-revisions <이전 리비전 이름>=100
gcloud run services update-traffic friend-agent --region us-central1 --to-latest
```

과제 2. 부하 테스트로 동시성과 최대 인스턴스 확인하기

섹션 4와 5에서 최대 인스턴스와 동시성을 설정으로만 바꿔 봤습니다. 이번에는 요청을 한꺼번에 보내는 부하 테스트(load test)로, 두 설정에 따라 인스턴스가 실제로 몇 개까지 늘어나는지와 처리하지 못한 요청이 어떻게 되는지 직접 봅니다.

에이전트 서비스에 부하 테스트를 하면 요청마다 LLM API가 호출되어 짧은 시간에 회사 키의 비용과 요청 한도를 써 버립니다. 그래서 LLM을 부르지 않는 테스트용 서비스 `load-test` 를 따로 만듭니다. 템플릿의 `hello/` 앱에 2초 기다렸다 응답하는 `/slow` 주소를 하나 추가해서, LLM 응답을 기다리느라 요청 하나가 몇 초

씩 걸리는 에이전트 서비스와 비슷한 상황을 만듭니다. 응답이 곧바로 끝나는 주소로 테스트하면 인스턴스 하나가 요청을 다 처리해 버려서 인스턴스가 늘어나는 것을 볼 수 없습니다.

코딩 에이전트에게 맡길 때: 과제 2: 부하 테스트용 서비스 만들기

```
hello/main.py 에 2초 기다렸다가 {"status": "ok"} 를 돌려주는 GET /slow 주소를 추가해 줘.
기다리는 동안 다른 요청을 막지 않도록 async 함수와 asyncio.sleep 으로 만들어 줘.
그다음 hello/ 폴더를 load-test 라는 이름의 Cloud Run 서비스로 us-central1 에 소스 배포해 줘. 로그인 없이 호출할 수 있게 하고, 최대 인스턴스는 3으로 해 줘.
```

배포가 끝나면 서비스 URL 끝에 `/slow` 를 붙여 브라우저로 열고, 2초쯤 뒤에 `{"status":"ok"}` 가 보이는지 확인합니다. 그다음 아래 스크립트를 저장소 루트에 `load_test.py` 로 저장합니다. Python 표준 라이브러리만 쓰므로 따로 설치할 것이 없습니다.

부하 테스트 스크립트 (`load_test.py`)

```
"""서비스 URL에 동시 요청을 보내고 응답 코드와 응답 시간을 요약한다.
사용법: python load_test.py <서비스 URL>/slow 50
"""
import sys
import time
import urllib.error
import urllib.request
from concurrent.futuresimport ThreadPoolExecutor

url= sys.argv[1]
total=int(sys.argv[2])iflen(sys.argv)>2else50


defsend(_):
    start= time.time()
try:
with urllib.request.urlopen(url, timeout=120)as response:
            code=str(response.status)
except urllib.error.HTTPErroras error:
        code=str(error.code)
except Exception:
        code="연결 실패"
return code, time.time()- start


with ThreadPoolExecutor(max_workers=total)as pool:
    results=list(pool.map(send,range(total)))

counts={}
for code, _in results:
    counts[code]= counts.get(code,0)+1
times=sorted(secondsfor _, secondsin results)
print(f"보낸 요청:{total}개")
print("응답 코드별 개수:", counts)
print(f"응답 시간: p50{times[len(times)//2]:.1f}초, p95{times[int(len(times)*0.95)-1]:.1f}초, 최대{times[-1]:.1f}초")
```

직접 실행할 명령: 과제 2: 세 가지 설정으로 부하 테스트

```
# 설정 A. 동시성 80(기본값), 최대 인스턴스 3
gcloud run services update load-test --region us-central1 --concurrency 80 --max-instances 3
python load_test.py <서비스 URL>/slow 50        # macOS에서 python 이 없으면 python3
```

```
# 설정 B. 동시성 1, 최대 인스턴스 20
gcloud run services update load-test --region us-central1 --concurrency 1 --max-instances 20
python load_test.py <서비스 URL>/slow 50

# 설정 C. 동시성 1, 최대 인스턴스 3
gcloud run services update load-test --region us-central1 --concurrency 1 --max-instances 3
python load_test.py <서비스 URL>/slow 50

# 끝나면 테스트용 서비스를 지운다
gcloud run services delete load-test --region us-central1
```

스크립트는 응답 코드별 개수와 응답 시간(p50, p95, 최대)을 출력합니다. 설정마다 결과를 아래 표에 옮기고, 콘솔의 Cloud Run > `load-test` > 측정항목(Metrics) 탭에서 컨테이너 인스턴스 수 그래프도 함께 봅니다. 설정을 바꾼 직후에는 새 리비전으로 넘어가는 데 몇십 초가 걸리므로, 리비전 탭에서 새 리비전이 트래픽 100%를 받는 것을 확인한 뒤 스크립트를 실행합니다.

설정	200 개수	200이 아닌 응답	p50	최대	최대 인스턴스 수 (그래프)
A. 동시성 80, 최대 3					
B. 동시성 1, 최대 20					
C. 동시성 1, 최대 3					

표를 채운 뒤 세 가지를 생각해 봅니다. 설정 A에서 인스턴스가 거의 늘지 않았다면, 인스턴스 하나가 요청 50개를 동시에 처리할 수 있었기 때문입니다. 설정 B에서 인스턴스가 여러 개로 늘었다면, 새 인스턴스가 켜지는 콜드 스타트 시간이 응답 시간에 어떻게 나타났는지 봅니다. 설정 C에서 200이 아닌 응답(429 등)이 나오거나 응답 시간이 크게 늘었다면, 최대 인스턴스는 비용을 막는 대신 처리하지 못한 요청을 기다리게 하거나 거절한다는 뜻입니다. 200이 아닌 응답은 섹션 6의 방법으로 `load-test` 의 요청 로그에서 상태 코드를 찾아 확인합니다. 에이전트 서비스는 요청 하나가 LLM 응답을 기다리는 동안 CPU를 거의 쓰지 않습니다. 그렇다면 에이전트 서비스의 동시성은 높게 두는 편이 맞을지 낮게 두는 편이 맞을지, 오늘 측정한 결과를 근거로 정해 보세요.

과제 3. 내가 만든 앱을 Cloud Run에 올리기

이전에 만든 FastAPI나 Streamlit 앱 폴더에서 소스 배포를 해 봅니다. 배포하기 전에 시작 명령이 `PORT` 환경변수를 쓰는지, LLM API 키 같은 시크릿이 코드에 들어 있지 않은지 먼저 확인합니다. 배포에 실패하면 섹션 6에서 한 것처럼 로그 종류를 나눠 타임라인을 만들고, 근거가 된 로그 줄을 붙여 에이전트에게 원인을 묻습니다. 서비스 URL이 열리고, 실패했다면 원인을 로그 줄과 함께 기록표에 적었으면 끝입니다.

코딩 에이전트에게 맡길 때: 과제 3

```
이 폴더의 앱을 Cloud Run에 소스 배포하려고 해. 배포 전에 세 가지를 먼저 확인해서 알려 줘.
1. 시작 명령이 PORT 환경변수의 포트를 리슨하는가
2. API 키 같은 시크릿이 코드나 커밋된 파일에 들어 있지 않은가 (키 값은 출력하지 마)
3. 필요한 라이브러리가 requirements.txt 에 모두 적혀 있는가
문제가 없으면 서비스 이름 my-app, 리전 us-central1 로 배포해 줘. 실패하면 고치지 말고 로그를 근거로 원인부터 설명해 줘.
```

과제 4. BYOK 구조로 바꾸기

섹션 5의 첫 번째 구조(브라우저에 두고 요청마다 보내기)로 정하고, 사용자가 자기 LLM API 키를 넣어 대화하게 바꿉니다. 키가 없는 사용자에게는 운영자 키를 쓰지 않고 키를 넣으라는 안내를 보여 줍니다. 에이전트가 설명한 계획에서 키가 지나가는 곳을 하나씩 따라가 보고, 저장소나 로그에 쓰는 코드가 있으면 승인하기 전에 빼 달라고 합니다. 로컬 환경에서 확인한 뒤 배포하고, 내 키를 넣으면 답이 오고 키를 지우면 안내가 나오는지 봅니다. 마지막으로 로그 탭 검색 칸에 내 키의 앞부분을 넣어 아무 줄도 나오지 않는지 확인하고, 검색 칸을 비웁니다.

코딩 에이전트에게 맡길 때: 과제 4

```
사용자가 화면에서 자기 LLM API 키를 넣고 그 키로 대화하게 바꾸고 싶어. 구조는 이렇게 정했어.
- 키는 브라우저의 sessionStorage 에만 두고, /api/chat 요청마다 X-User-LLM-Key 헤더로 보낸다.
- 서버는 그 요청을 처리하는 동안만 키를 쓰고, 저장소, 로그, 오류 메시지, 이번 턴 패널 어디에도 남기지 않는다.
- 헤더가 없으면 운영자 키를 쓰지 말고, 키를 넣으라는 안내를 돌려준다.
- 화면에는 키 입력 칸, 넣은 키의 끝 네 자리, 키 지우기 버튼을 둔다.
- 키가 틀렸거나 한도에 닿아 LLM 제공사가 거부하면 그 이유를 화면에 안내한다.
코드를 고치기 전에 바꿀 파일과 각 파일에서 바꿀 내용을 먼저 설명해 줘.
```

Chapter08

오늘 배운 것

시간과 개수. AI 서비스의 요청은 답을 보낸 뒤 기억 정리까지 같은 요청 안에서 끝나므로, 요청 타임아웃은 측정한 요청 전체 시간보다 넉넉하게 잡습니다. 최대 인스턴스는 한 번에 처리하는 양을 막을 뿐이고, 최소 인스턴스를 0으로 두면 요금 대신 콜드 스타트를 받아들입니다.

데이터와 비용. 인스턴스 안에 둔 대화는 배포나 인스턴스 교체 때 사라집니다. 비용의 대부분은 Cloud Run이 아니라 LLM API 호출에서 나오고, 총액을 막는 것은 LLM 제공사 쪽 사용 한도입니다. 예산 알림은 알려 주기만 합니다.

로그 읽기. 로그 탭에는 요청 로그, 컨테이너 로그, 시스템 로그가 섞여 있고, 빌드 로그는 Cloud Build에 따로 있습니다. 심각도, 상태 코드, 지연 시간, 리비전으로 줄을 거르고, 흩어진 줄을 시간순으로 모으면 무슨 일이 있었는지 다시 세울 수 있습니다. 터미널 메시지는 대개 결과를 말하고, 원인은 그보다 앞선 컨테이너 로그에 있었습니다. 에이전트에게는 증상, 시간 범위, 서비스와 리비전, 찾은 로그 줄을 근거로 건넸습니다.

주관식 퀴즈

Q. [회고] 섹션 1에서 공개 주소를 퍼뜨렸을 때 걱정되는 일을 세 가지 적었습니다. 오늘 건 설정 가운데 각각을 막아 주는 것과, 아직 막지 못하는 것을 나눠 적어 보세요.